COMPAS Data Exploration & Interactive Dashboard

Usaydh Habib uhabib@umich.edu GitHub: Dashboard: Video:

Introduction

I have always been interested in how data and algorithms are used to make real-world decisions. Today, I will be diving into a dataset about COMPAS risk scores and criminal recidivism. COMPAS is a software tool used by courts to predict whether a defendant will reoffend.

In this tutorial, I am going to build an interactive dashboard using Python. The goal is to show how four different chart types—a Bar Chart, a Histogram, a Scatter Plot, and a Box Plot—can work together to help explore potential biases in risk scoring.

Visualization Technique

To understand this dataset completely, I combined four visual chart types into a 2x2 grid dashboard:

Bar Chart (Recidivism Rate by Race): Displays the average percentage of defendants who reoffended within two years, broken down by race.

Histogram (Decile Score Spread): Shows the overall distribution of COMPAS decile scores ranging from 1 (lowest risk) to 10 (highest risk).

Scatter Plot (Priors Count vs. Age): Plots each defendant by their age on the X-axis and their total count of prior convictions on the Y-axis. Points are color-coded by their decile score.

Box Plot (Decile Scores by Charge Severity): Displays the median, quartiles, and range of COMPAS risk scores across Felony and Misdemeanor charges.

Visual Complementarity & Interactivity Considerations

A single chart only tells one side of the story. If we only looked at a bar chart of recidivism, we wouldn't see how age or prior convictions influence the risk score.

By placing all four graphs on one screen, users can filter the data by race, age group, or charge degree and immediately watch all four graphs update at the same time. Using dynamic filtering allows us to ask specific questions and get visual feedback across all charts.

Visualization Library

For this assignment, I am using Plotly and ipywidgets.

Plotly was created by Plotly Inc. as an open-source graphing library built on Plotly.js

Plotly and ipywidgets are open-source and free to use under the MIT license.

How to Install: pip install plotly ipywidgets pandas

Why avoid Matplotlib, Seaborn, or Altair?

Matplotlib & Seaborn: These libraries generate static images. They are great for reports but they do not support live interactive sliders or multi-chart linked callbacks inside notebooks out of the box.

Altair: Altair has a strict default limit of 5,000 rows. The COMPAS dataset has over 7,200 rows. This causes Altair to throw errors or render very slowly.

Plotly's Advantage: Plotly includes WebGL support through go.Scattergl. This uses hardware acceleration on the graphics card to render thousands of scatter plot data points quickly.

Dataset & Cleaning Instructions

We are using the compas-scores-two-years.csv dataset.

Cleaning Steps:

Load the raw dataset and filter for essential columns (id, age, sex, race, priors_count, c_charge_degree, decile_score, two_year_recid).

Remove missing or empty rows using .dropna().

Map charge degrees ('F' and 'M') to readable text ('Felony' and 'Misdemeanor').

Map numerical recidivism flags (0 and 1) to clear labels ('No Recidivism' and 'Recidivated').

In [20]:
```
import pandas as pd
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import ipywidgets as widgets
from ipywidgets import interact

# 1. Load Data
df = pd.read_csv('compas-scores-two-years.csv')

# 2. Clean Data
cols = ['id', 'age', 'age_cat', 'sex', 'race', 'priors_count',
        'c_charge_degree', 'decile_score', 'two_year_recid']
df_clean = df[cols].dropna().copy()

# Focus on major demographic groups for clear comparison
df_clean = df_clean[df_clean['race'].isin(['African-American', 'Caucas

# Readable mapping for legend labels
df_clean['recid_label'] = df_clean['two_year_recid'].map({0: 'No Recid
df_clean['charge_label'] = df_clean['c_charge_degree'].map({'F': 'Felo

print("Cleaned Data shape:", df_clean.shape)
df_clean.head()
```

```
Cleaned Data shape: (7164, 11)
```

Out[20]:

	id	age	age_cat	sex	race	priors_count	c_charge_degree	decile_sc
0	1	69	Greater than 45	Male	Other	0	F	
1	3	34	25 - 45	Male	African-American	0	F	
2	4	24	Less than 25	Male	African-American	4	F	
3	5	23	Less than 25	Male	African-American	1	F	
4	6	43	25 - 45	Male	Other	2	F	

Dashboard Source Code

Here is the complete code cell that builds the interactive widgets and creates all 4 chart types together.

In [23]:
```
# 3. Dynamic Interactive Dashboard Function
@interact(
    races=widgets.SelectMultiple(
        options=list(df_clean['race'].unique()),
        value=list(df_clean['race'].unique()),
        description='Race:',
        disabled=False
    ),
    age_range=widgets.IntRangeSlider(
        value=[18, 70],
        min=int(df_clean['age'].min()),
        max=int(df_clean['age'].max()),
        step=1,
        description='Age Range:'
    ),
    charge=widgets.Dropdown(
        options=['All', 'Felony', 'Misdemeanor'],
        value='All',
        description='Charge:'
    )
)
def generate_dashboard(races, age_range, charge):
    # Filter Dataset based on widget choices
    filtered = df_clean[
        (df_clean['race'].isin(races)) &
        (df_clean['age'] >= age_range[0]) &
        (df_clean['age'] <= age_range[1])
    ]
    if charge != 'All':
        filtered = filtered[filtered['charge_label'] == charge]

    # Create 2x2 Subplot Layout
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=(
            '1. Recidivism Rate by Race (Bar Chart)',
            '2. Decile Score Distribution (Histogram)',
            '3. Priors Count vs. Age (Scatter Plot)',
            '4. Decile Score by Charge (Box Plot)'
        )
    )

    # Visual 1: Bar Chart
    recid_summary = filtered.groupby('race')['two_year_recid'].mean().
    fig.add_trace(
        go.Bar(
            x=recid_summary['race'],
            y=recid_summary['two_year_recid'],
            name='Recidivism Rate',
```

```
            marker_color='indigo'
        ),
        row=1, col=1
    )

    # Visual 2: Histogram
    fig.add_trace(
        go.Histogram(
            x=filtered['decile_score'],
            name='Decile Score',
            marker_color='teal',
            nbinsx=10
        ),
        row=1, col=2
    )

    # Visual 3: Scatter Plot (using Scattergl for high performance)
    fig.add_trace(
        go.Scattergl(
            x=filtered['age'],
            y=filtered['priors_count'],
            mode='markers',
            name='Age vs Priors',
            marker=dict(
                color=filtered['decile_score'],
                colorscale='Viridis',
                showscale=True
            )
        ),
        row=2, col=1
    )

    # Visual 4: Box Plot
    fig.add_trace(
        go.Box(
            x=filtered['charge_label'],
            y=filtered['decile_score'],
            name='Charge Severity',
            marker_color='coral'
        ),
        row=2, col=2
    )

    # Layout Setup
    fig.update_layout(
        height=750,
        width=950,
        title_text="COMPAS Risk & Recidivism Dynamic Dashboard",
        showlegend=False
    )

    # Axis Titles
```

```
    fig.update_xaxes(title_text="Race", row=1, col=1)
    fig.update_yaxes(title_text="Recidivism Rate (0-1)", row=1, col=1)
    fig.update_xaxes(title_text="COMPAS Decile Score (1-10)", row=1, c
    fig.update_yaxes(title_text="Count", row=1, col=2)
    fig.update_xaxes(title_text="Age", row=2, col=1)
    fig.update_yaxes(title_text="Priors Count", row=2, col=1)
    fig.update_xaxes(title_text="Charge Severity", row=2, col=2)
    fig.update_yaxes(title_text="COMPAS Decile Score", row=2, col=2)

    fig.show()
```

```
interactive(children=(SelectMultiple(description='Race:', index=(0, 1,
2, 3), options=('Other', 'African-Ameri…
```

In [ ]: